# ORDERLY

## Technical Design Document (TDD)

- **Project Name:** Orderly
- **Document Type:** Technical Design Document
- **Domain:** Fast-Moving Consumer Goods (FMCG) B2B Supply Chain & Automated Order Capture
- **Author:** Vedant Kolte
- **Version:** 1.0.0 (Production Release & Internship Evaluation Submission)
- **Document Date:** November 9, 2026
- **Project Repository:** https://github.com/ved-2/orderly
- **Live Backend Deployment:** https://orderly-w9x4.onrender.com
- **Frontend Live Deployment:** https://orderly-eight-silk.vercel.app/

## Executive Summary

Orderly is an enterprise-grade B2B ordering and distribution operations platform designed specifically for regional FMCG distributors and their neighborhood kirana retail networks in tier-2 and tier-3 Indian markets. Regional distributors in India operate in high-volume, low-margin environments where order capture has historically relied on manual, informal, and fragmented channels—primarily physical paper notebooks carried by field salesmen, disorganized WhatsApp text messages, low-light photographs of handwritten paper slips (*parchis*), and vernacular audio voice notes.

This reliance on informal communication introduces critical operational bottlenecks: back-office staff must manually decipher handwriting and audio notes, cross-reference unstandardized colloquial product descriptions against thousands of master Stock Keeping Units (SKUs), manually calculate multi-tiered wholesale trade schemes, verify retailer credit limits, and key transactions into accounting systems like Tally ERP. When stock-outs or scheme miscalculations occur at the point of ordering, order lines must be cut later during dispatch, triggering delivery disputes, high return rates, stock write-downs, and lost gross margin.

Orderly resolves these structural challenges through a modern, role-based, decoupled web platform paired with a hybrid multilingual AI order capture engine:

1. **Kirana Retailer Portal:** An installable, lightweight Progressive Web App (PWA) optimized for low-end mobile devices and 3G networks. It provides real-time catalog visibility, retailer-specific wholesale pricing with explicit GST breakdowns, automated scheme application, live credit limit status, one-click reordering, and a conversational AI ordering assistant supporting English, Hindi, and colloquial Hinglish.

2. **Back-Office Order Desk:** A central operations console providing real-time triage of incoming orders, credit-hold enforcement for over-limit retailers, atomic invoice approval workflows, one-click compliant Tally ERP XML exports, vehicle van loading sheets batched by FMCG brand, and automated alerts identifying dormant retailers (zero orders in 14+ days).

3. **Field Salesman Beat Mode:** An operational interface enabling territory sales representatives to manage assigned retail beats, log store visits, review merchant outstanding balances at the physical counter, and capture orders in offline-tolerant workflows.

4. **Distributor Admin Portal:** A management dashboard providing master catalog controls, multi-tier pricing configurations, dynamic trade scheme builders (Buy X Get Y Free and volume slab discounts), credit ledger management, salesman territory assignments, staff invitation governance, and sales revenue analytics.

## Project Scope

The project was developed within a 7-day solo engineering sprint. To ensure quality and reliable end-to-end functionality, the project scope was divided into four categories: implemented core features, additional implemented features, partially implemented features, and out-of-scope features.

### Implemented Core Functionality

The following features form the primary working functionality of Orderly.

| Feature | Description |
|---|---|
| Authentication and RBAC | Firebase Authentication with PostgreSQL-based role mapping for Distributor Admin, Order Desk, Salesman, and Retailer. |
| Product Catalogue | Product and brand management with unique SKUs, packaging details, HSN codes, and GST rates. |
| Wholesale Pricing | Default distributor pricing with retailer-specific price overrides and validity dates. |
| Scheme Engine | Calculates Buy X Get Y free-goods schemes and slab-based percentage discounts using deterministic business rules. |
| Credit Management | Maintains credit balances, transaction history, and automatic order holds when credit limits are exceeded. |
| Order Lifecycle | Manages orders through Draft, Submitted, Under Review, On Hold, Approved, Rejected, Dispatched, and Cancelled states. |

| | |
|---|---|
| Order Audit History | Maintains order status history for tracking changes and approval activities. |
| AI Order Capture | Uses Groq to extract products, quantities, units, and order intent from natural-language messages, including Hindi number words. |
| Fallback Order Parser | Uses deterministic regex and phonetic matching when AI parsing is unavailable or unsuccessful. |
| Order Disambiguation | Asks users to clarify missing units or select alternative products when product matching confidence is low. |

**Additional Implemented Functionality**

The following features extend the platform beyond its primary ordering workflow.

| **Feature** | **Description** |
|---|---|
| Handwritten Parchi OCR | Uses Gemini 2.5 Flash Vision to extract order details from uploaded handwritten order slips. |
| Tally ERP Integration | Generates XML files compatible with Tally ERP 9 and Tally Prime for importing order-related data. |
| Van Loading Sheets | Groups approved order quantities by product and brand to support warehouse packing and vehicle loading. |

| | |
|---|---|
| Dormant Retailer Detection | Identifies retailers who have not placed an approved order in the previous 14 days. |
| Salesman Territory Management | Maps salesmen to assigned retailers and geographic beats, with order placement and performance summaries. |
| Staff Invitations | Uses hashed, single-use invitation tokens with configurable expiration times for staff onboarding. |
| Sales Analytics | Provides distributor-level sales information and order summaries. |
| Bilingual Interface | Supports a bilingual user interface and transparent GST information. |

**Architectural Assumptions**

The following assumptions guided the system design and implementation. They represent project-level design decisions and should not be interpreted as verified requirements from the client.

| **ID** | **Assumption** | **Design Impact** |
|---|---|---|
| ASSUMPTION-01 | Retailers and salesmen may use entry-level Android smartphones with limited memory and inconsistent 3G/4G connectivity. | The interface should be responsive, lightweight, easy to navigate, and usable on smaller screens. |
| ASSUMPTION-02 | Wholesale orders are generally placed in full cases (peti), while individual | The system uses integer-based case-to-piece conversions and does not |

| | | |
|---|---|---|
| | pieces (packet/nag) are used for retail sales. | allow fractional cases in wholesale orders. |
| ASSUMPTION-03 | Trade schemes are treated as non-compoundable promotions, such as Buy X Get Y Free and volume-based slab discounts. | Scheme calculations use deterministic business rules and two-decimal monetary precision. |
| ASSUMPTION-04 | Retailers operate with assigned credit limits and may have rolling credit periods. | Orders that exceed the available credit limit are automatically placed on hold for manual review. |
| ASSUMPTION-05 | Tally ERP remains the primary system for statutory accounting and GST compliance. | Orderly generates Tally-compatible XML exports instead of directly integrating with the GST portal. |
| ASSUMPTION-06 | AI-generated order details must be reviewed before becoming confirmed orders. | AI extraction creates draft order items. Retailers or authorized staff must confirm the details before order approval. |

# USER ROLES AND ACCESS CONTROL

Orderly implements Role-Based Access Control (RBAC) enforced at both the API layer (via FastAPI dependency injection require_roles) and the frontend presentation layer (via Next.js layout route guards and contextual navigation).

## ROLE DEFINITIONS AND RESPONSIBILITIES

### 1. Distributor Administrator (DISTRIBUTOR_ADMIN)

- Profile: The business owner, managing director, or general manager of Company A.
- Responsibilities: Master catalog governance, price list updates, scheme creation, staff management, territory assignments, and global business health monitoring.
- Accessible Modules: All modules across the system (/distributor/*, /order-desk/*, /products/*, /schemes/*, /credit/*).
- Permitted Actions: Read and write all products, brands, and prices; create and toggle trade schemes; issue and revoke staff invitations; assign retailers to salesmen; override credit limits; view comprehensive sales revenue metrics; approve, put on hold, or reject orders.
- Restricted Actions: Cannot place orders as a retailer directly without selecting a target retailer context.

### 2. Order Desk Operator (ORDER_DESK)

- Profile: Back-office administrative personnel working in the central Jabalpur warehouse office.
- Responsibilities: Review incoming order queues, inspect flagged orders, resolve credit holds, print van loading sheets, and export approved batches to Tally ERP.
- Accessible Modules: Order Desk workspace (/order-desk), order review queues, credit account inspector, Tally XML exporter, van loading sheets, and dormant retailer reports.
- Permitted Actions: View all submitted orders; update order statuses (UNDER_REVIEW, APPROVED, ON_HOLD, REJECTED, DISPATCHED); inspect retailer credit ledgers; download Tally ERP XML export batches; generate van loading sheets; view dormant retailer reports.
- Restricted Actions: Cannot delete catalog items, alter global price master lists, or modify system staff configurations.

### 3. Field Salesman (SALESMAN)

- Profile: On-ground territory sales representatives visiting 30 to 40 kirana shops daily across Jabalpur.
- Responsibilities: Conduct shop counter visits, verify stock needs, review merchant credit limits on-site, and book orders on behalf of assigned retailers.

- Accessible Modules: Salesman Beat Portal (/salesman), assigned retailer list, product catalog, and daily beat summary.
- Permitted Actions: View assigned kirana retailers on designated beat schedules; access live product catalogs and active schemes; submit orders on behalf of assigned retailers (proxy ordering); view personal daily beat summaries and sales performance.
- Restricted Actions: Cannot access unassigned retailer ledgers, alter master prices, or approve/dispatch orders.

**4. Kirana Retailer (RETAILER)**

- Profile: Neighborhood kirana shop owners placing replenishment orders directly.
- Responsibilities: Browse wholesale stock, check store pricing, review active trade schemes, submit orders via catalog or conversational AI, and track order fulfillment.
- Accessible Modules: Retailer Portal (/retailer/*) including Dashboard, Product Catalog, Cart, Checkout, Conversational AI Assistant, Order History, Credit Ledger, and Shop Settings.
- Permitted Actions: Access active product catalog with personalized pricing; manage personal cart and submit orders; interact with AI conversational assistant and parchi scanner; view personal order history, credit limit, and ledger balance; update shop settings.
- Restricted Actions: Completely isolated from other retailers' data; cannot view distributor margins, global sales reports, other merchants' credit terms, or order desk queues.

**Architectural Data Flow**

The following steps describe how requests move through the system, from the frontend to the database and external services.

**1. Client Interaction and Input Processing**

Retailers, salesmen, and administrators access Orderly through their respective portal interfaces. The frontend provides responsive interfaces for product browsing, order placement, order management, and administrative operations.

For voice input, the browser uses the Web Audio API with a bandpass filter configured between 300 Hz and 3.4 kHz to reduce unwanted background noise before speech processing.

**2. API Gateway and Authentication**

All frontend requests are sent to the FastAPI backend through the versioned REST API.

The authentication process works as follows:

- The frontend sends a Firebase Bearer token with the API request.
- The backend extracts the token through the get_current_user dependency.
- Firebase Admin SDK verifies the token's signature and expiration.
- The backend identifies the corresponding user in the PostgreSQL database.
- The require_roles dependency checks whether the user has permission to access the requested operation.
- Unauthorized requests are rejected with an appropriate HTTP 403 response.

Pydantic models validate incoming request data before it reaches the business logic.

**3. Business Logic and Order Processing**

When a retailer creates an order, the backend processes it through multiple services.

| **Processing Stage** | **Service** | **Operation** |
|---|---|---|
| Price Resolution | PricingService | Retrieves retailer-specific pricing or falls back to the distributor's base price. |

| | | |
|---|---|---|
| Scheme Calculation | SchemeService | Applies eligible free-goods schemes and volume-based discounts. |
| Credit Validation | CreditService | Checks available credit against the order value and assigned credit limit. |
| Order Management | OrderService | Creates the order and manages its lifecycle and status transitions. |
| Audit Recording | Order History | Records order status changes for traceability. |

If the order exceeds the retailer's available credit limit, it is placed in the ON_HOLD state for manual review.

Database transactions and row-level locking are used where required to maintain consistency during critical credit operations.

**AI Processing Pipeline**

The processing pipeline follows these steps:

1. The user submits an order through text, voice transcription, or an uploaded image.
2. The backend forwards the input to the appropriate processing service.
3. The AI service extracts product names, quantities, and relevant units.
4. Extracted product names are matched against the product catalogue.
5. Uncertain or incomplete matches are sent through the disambiguation workflow.
6. If an external AI service fails, the deterministic fallback parser is used where supported.
7. The extracted items are presented for user confirmation before the order proceeds.

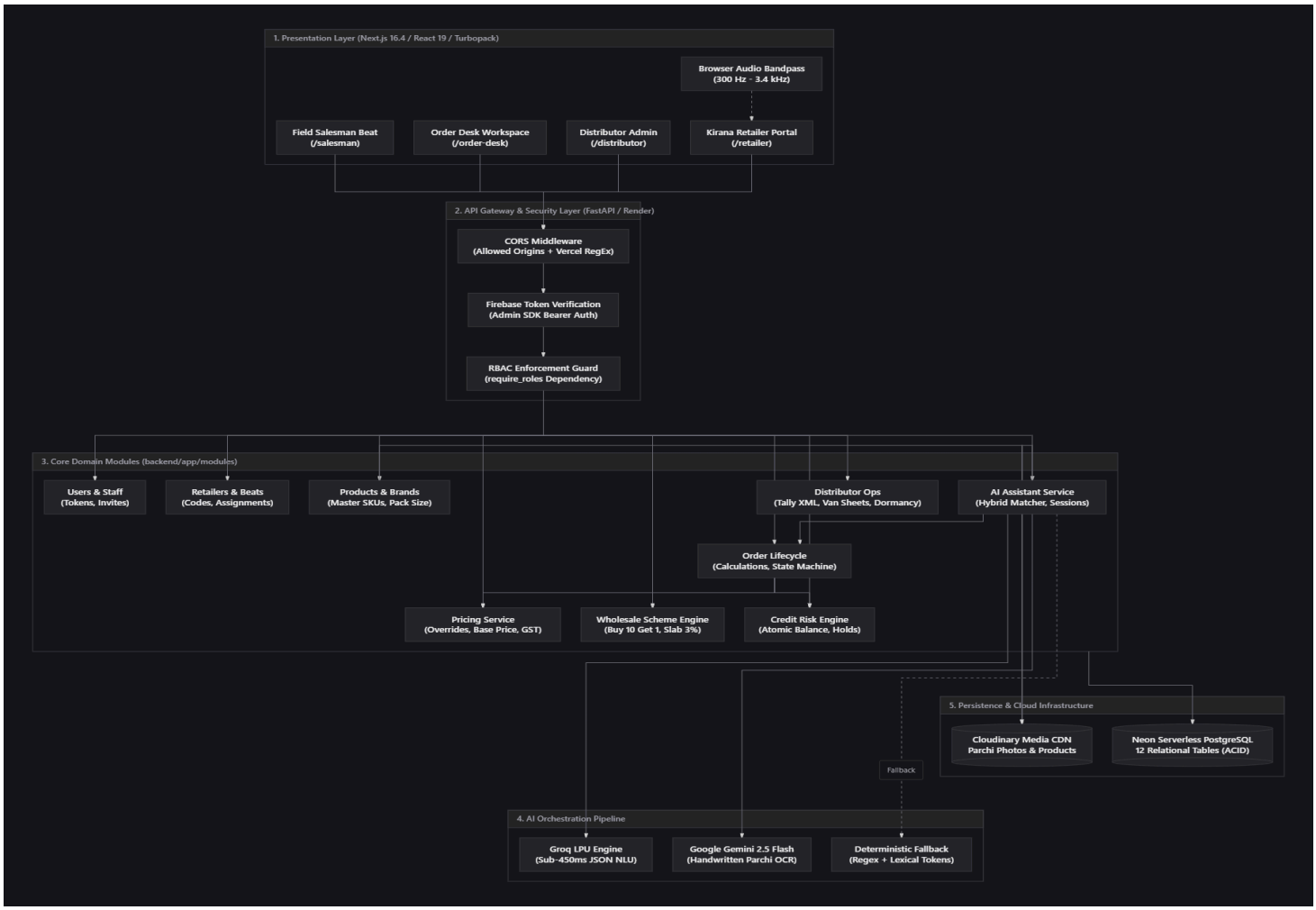
AI-generated results are not treated as automatically approved orders. Human confirmation is required before the order enters the normal approval workflow.

**Database Persistence and Accounting Export**

After validation and processing, application data is stored in Neon Serverless PostgreSQL.

The database maintains records for users, retailers, products, pricing, schemes, orders, and credit transactions. SQLAlchemy handles database operations, while Alembic manages schema migrations.

The Order Desk can generate Tally-compatible XML exports for approved orders. These files can then be imported into the distributor's Tally ERP 9 or Tally Prime desktop application.



# DATABASE DESIGN

The persistence layer is modeled in PostgreSQL 16 using SQLAlchemy 2.0 and Alembic. The schema adheres to Third Normal Form (3NF) principles, enforcing relational integrity, explicit check constraints, composite unique indexes, and double-entry ledgering.

**DETAILED SCHEMA CATALOG, CONSTRAINTS & DATA RULES**

**1. Table: users**

- id: UUID (Primary Key, default uuid.uuid4).
- firebase_uid: VARCHAR(128) (Unique, Indexed, Nullable=False). Links to Firebase Identity.
- email: VARCHAR(255) (Unique, Indexed, Nullable=False).
- phone: VARCHAR(20) (Unique, Nullable=True).
- role: VARCHAR(30) (Nullable=False). Values: DISTRIBUTOR_ADMIN, ORDER_DESK, SALESMAN, RETAILER.
- is_active: BOOLEAN (Default=True, Nullable=False).
- created_at / updated_at: DateTime(timezone=True) (Server Default func.now()).

**2. Table: staff_invitations**

- id: UUID (Primary Key).
- email: VARCHAR(255) (Indexed, Nullable=False). Target staff email.
- role: VARCHAR(30) (Nullable=False). Role to assign upon redemption.
- token_hash: VARCHAR(128) (Unique, Indexed, Nullable=False). SHA-256 hash of the random 32-byte secret invite token.
- expires_at: DateTime(timezone=True) (Nullable=False). Expiration threshold.
- used_at: DateTime(timezone=True) (Nullable=True). Timestamp when redeemed; prevents reuse.
- invited_by: UUID (Foreign Key referencing users.id ondelete RESTRICT).

3. **Table: retailers**

- id: UUID (Primary Key).
- retailer_code: VARCHAR(30) (Unique, Indexed, Nullable=False, e.g. RET-JBP-01).
- shop_name: VARCHAR(150) (Nullable=False). Commercial enterprise name.
- owner_name: VARCHAR(120) (Nullable=False). Proprietor name.
- phone: VARCHAR(20) (Nullable=False). Contact number.
- address: VARCHAR(300) (Nullable=False). Physical counter location.
- city: VARCHAR(100) (Nullable=False, e.g. Jabalpur).

- state: VARCHAR(100) (Nullable=False, default Madhya Pradesh).
- pincode: VARCHAR(10) (Nullable=False). Postal code.
- is_active: BOOLEAN (Default=True, Nullable=False).
- registration_status: VARCHAR(30) (Default PENDING_REVIEW, Indexed).
- Check Constraint (ck_retailer_registration_status): registration_status IN ('PENDING_REVIEW', 'ACTIVE', 'REJECTED', 'SUSPENDED').
- duplicate_flag: BOOLEAN (Default=False). Set if phone or name suggests duplicate.

**4. Table: retailer_users**

- id: UUID (Primary Key).
- retailer_id: UUID (Foreign Key referencing retailers.id ondelete CASCADE).
- user_id: UUID (Foreign Key referencing users.id ondelete CASCADE).
- Unique Constraint (uq_retailer_user): (retailer_id, user_id). Connects login credentials to a shop entity.

**5. Table: retailer_assignments**

- id: UUID (Primary Key).
- retailer_id: UUID (Foreign Key referencing retailers.id ondelete CASCADE).
- salesman_id: UUID (Foreign Key referencing users.id ondelete CASCADE).
- Unique Constraint (uq_retailer_salesman): (retailer_id, salesman_id). Represents salesman beat allocations.

**6. Table: retailer_settings**

- id: UUID (Primary Key).
- retailer_id: UUID (Foreign Key referencing retailers.id ondelete CASCADE, Unique, Indexed).
- gstin: VARCHAR(20) (Nullable=True). Optional GSTIN number.
- language: VARCHAR(10) (Default 'en', Nullable=False). Values: en, hi.
- Notification Flags: order_notifications, order_confirmation_notifications, delivery_notifications, account_notifications, promotional_notifications (BOOLEAN, Default=True).

**7. Table: brands**

- id: UUID (Primary Key).
- name: VARCHAR(100) (Unique, Nullable=False). FMCG brand name.

- code: VARCHAR(30) (Unique, Nullable=False). Short brand code.
- is_active: BOOLEAN (Default=True).

**8. Table: products**

- id: UUID (Primary Key).
- brand_id: UUID (Foreign Key referencing brands.id ondelete RESTRICT, Indexed).
- sku: VARCHAR(50) (Unique, Indexed, Nullable=False, e.g. SKU-BS-075).
- name: VARCHAR(150) (Nullable=False). Official trade name.
- description: TEXT (Nullable=True).
- category: VARCHAR(80) (Nullable=False, e.g. Packaged Foods, Home Care).
- pack_size: NUMERIC(12, 3) (Nullable=False, e.g. 75.000). Weight/volume per single piece.
- pack_unit: VARCHAR(20) (Nullable=False, e.g. gm, kg, ml).
- units_per_case: INTEGER (Nullable=False, e.g. 96). Packaging conversion ratio.
- hsn_code: VARCHAR(20) (Nullable=True). Statutory tax classification code.
- gst_rate: NUMERIC(5, 2) (Nullable=True, e.g. 18.00). Applicable GST percentage.
- is_active: BOOLEAN (Default=True).

**9. Table: product_aliases**

- id: UUID (Primary Key).
- product_id: UUID (Foreign Key referencing products.id ondelete CASCADE, Indexed).
- alias: VARCHAR(150) (Nullable=False). Vernacular or colloquial product phrase (e.g. "chhota biscuit").
- normalized_alias: VARCHAR(150) (Indexed, Nullable=False). Lowercase, stripped tokens.
- language: VARCHAR(10) (Default 'en', Nullable=False).
- Unique Constraint (uq_product_alias_language): (product_id, normalized_alias, language).

**10. Table: distributor_product_prices (Default Wholesale Pricing Baseline)**

- id: UUID (Primary Key).
- product_id: UUID (Foreign Key referencing products.id ondelete RESTRICT, Indexed).
- price: NUMERIC(12, 2) (Nullable=False). Baseline dealer piece price.
- gst_included: BOOLEAN (Default=True, Nullable=False).
- valid_from: DateTime(timezone=True) (Nullable=False).
- valid_to: DateTime(timezone=True) (Nullable=True).
- Check Constraints: price >= 0, valid_to IS NULL OR valid_to > valid_from.

- Unique Constraint (uq_distributor_product_price_start): (product_id, valid_from).

**11. Table: retailer_product_prices (Retailer-Specific Price Overrides)**

- id: UUID (Primary Key).
- retailer_id: UUID (Foreign Key referencing retailers.id ondelete RESTRICT, Indexed).
- product_id: UUID (Foreign Key referencing products.id ondelete RESTRICT, Indexed).
- price: NUMERIC(12, 2) (Nullable=False). Custom negotiated piece price.
- gst_included: BOOLEAN (Default=True, Nullable=False).
- valid_from: DateTime(timezone=True) (Nullable=False).
- valid_to: DateTime(timezone=True) (Nullable=True).
- Check Constraints: price >= 0, valid_to IS NULL OR valid_to > valid_from.
- Unique Constraint (uq_retailer_product_price_start): (retailer_id, product_id, valid_from).

**12. Table: schemes**

- id: UUID (Primary Key).
- code: VARCHAR(40) (Unique, Nullable=False). Short scheme code.
- name: VARCHAR(150) (Nullable=False). Human-readable commercial title.
- description: TEXT (Nullable=True).
- scheme_type: VARCHAR(30) (Nullable=False). Check Constraint: scheme_type IN ('BUY_X_GET_Y', 'SLAB_DISCOUNT').
- valid_from / valid_to: DateTime(timezone=True) (Nullable=False). Check Constraint: valid_to > valid_from.
- applies_to_all_retailers: BOOLEAN (Default=True, Nullable=False).
- is_active: BOOLEAN (Default=True, Nullable=False).

**13. Table: scheme_rules**

- id: UUID (Primary Key).
- scheme_id: UUID (Foreign Key referencing schemes.id ondelete CASCADE, Indexed).
- product_id: UUID (Foreign Key referencing products.id ondelete RESTRICT, Indexed).
- reward_product_id: UUID (Foreign Key referencing products.id ondelete RESTRICT, Nullable=True).
- tier_number: INTEGER (Default=1, Nullable=False). Check Constraint: tier_number >= 1.
- quantity_unit: VARCHAR(10) (Default 'CASE', Nullable=False). Check Constraint: quantity_unit IN ('CASE', 'PIECE').

- buy_quantity: NUMERIC(12, 3) (Nullable=True). Threshold needed for free goods.
- reward_quantity: NUMERIC(12, 3) (Nullable=True). Number of free units awarded.
- min_quantity: NUMERIC(12, 3) (Nullable=True). Volume threshold for slab discount.
- discount_type: VARCHAR(20) (Nullable=True). Check Constraint: discount_type IN ('PERCENTAGE', 'FIXED_AMOUNT').
- discount_value: NUMERIC(12, 2) (Nullable=True). Percentage or rupee amount.
- Check Constraint (ck_scheme_percentage_limit): discount_type != 'PERCENTAGE' OR discount_value <= 100.
- Unique Constraint (uq_scheme_product_tier): (scheme_id, product_id, tier_number).

**14. Table: credit_accounts**

- id: UUID (Primary Key).
- retailer_id: UUID (Foreign Key referencing retailers.id ondelete RESTRICT, Unique, Nullable=False).
- credit_limit: NUMERIC(14, 2) (Default=0, Nullable=False). Check Constraint: credit_limit >= 0.
- current_outstanding: NUMERIC(14, 2) (Default=0, Nullable=False). Check Constraint: current_outstanding >= 0.
- payment_terms_days: INTEGER (Default=7, Nullable=False).
- updated_at: DateTime(timezone=True) (Server Default func.now(), onupdate func.now()).

**15. Table: credit_transactions**

- id: UUID (Primary Key).
- credit_account_id: UUID (Foreign Key referencing credit_accounts.id ondelete RESTRICT, Indexed).
- transaction_type: VARCHAR(30) (Nullable=False). Check Constraint: transaction_type IN ('ORDER_DEBIT', 'PAYMENT', 'CREDIT_NOTE', 'ADJUSTMENT').
- amount: NUMERIC(14, 2) (Nullable=False). Check Constraint: amount > 0.
- reference_type: VARCHAR(30) (Nullable=True). Usually "ORDER".
- reference_id: UUID (Nullable=True). Links to orders.id.
- note: VARCHAR(300) (Nullable=True).
- created_by: UUID (Foreign Key referencing users.id ondelete RESTRICT, Nullable=True).

**16. Table: orders**

- id: UUID (Primary Key).
- order_number: VARCHAR(40) (Unique, Indexed, Nullable=False). Format: ORD-YYMMDD-XXXX.

- retailer_id: UUID (Foreign Key referencing retailers.id ondelete RESTRICT, Indexed).
- created_by: UUID (Foreign Key referencing users.id ondelete RESTRICT).
- status: VARCHAR(30) (Default 'DRAFT', Indexed, Nullable=False).
- Check Constraint (ck_order_status): status IN ('DRAFT', 'SUBMITTED', 'UNDER_REVIEW', 'APPROVED', 'ON_HOLD', 'REJECTED', 'DISPATCHED', 'CANCELLED').
- source: VARCHAR(30) (Default 'RETAILER_WEB', Nullable=False).
- Check Constraint (ck_order_source): source IN ('RETAILER_WEB', 'SALESMAN', 'TELEGRAM', 'AI_CAPTURE').
- idempotency_key: VARCHAR(100) (Unique, Nullable=True). Protects against network duplicate orders.
- subtotal: NUMERIC(14, 2) (Default=0, Nullable=False).
- discount_amount: NUMERIC(14, 2) (Default=0, Nullable=False).
- tax_amount: NUMERIC(14, 2) (Default=0, Nullable=False).
- total_amount: NUMERIC(14, 2) (Default=0, Nullable=False).
- Check Constraint (ck_order_amounts_nonnegative): subtotal >= 0 AND discount_amount >= 0 AND tax_amount >= 0 AND total_amount >= 0.
- customer_note / desk_note: TEXT (Nullable=True).
- submitted_at / approved_at / dispatched_at: DateTime(timezone=True) (Nullable=True).

**17. Table: order_items**

- id: UUID (Primary Key).
- order_id: UUID (Foreign Key referencing orders.id ondelete CASCADE, Indexed).
- product_id: UUID (Foreign Key referencing products.id ondelete RESTRICT).
- scheme_id: UUID (Foreign Key referencing schemes.id ondelete SET NULL, Nullable=True).
- product_name: VARCHAR(150) (Nullable=False). Snapshot of product name at order time.
- sku: VARCHAR(50) (Nullable=False). Snapshot of SKU code at order time.
- quantity: NUMERIC(12, 3) (Nullable=False). Check Constraint: quantity > 0.
- quantity_unit: VARCHAR(10) (Nullable=False). Check Constraint: quantity_unit IN ('CASE', 'PIECE').
- free_quantity: NUMERIC(12, 3) (Default=0, Nullable=False). Check Constraint: free_quantity >= 0.
- unit_price: NUMERIC(12, 2) (Nullable=False). Price per unit.
- discount_amount: NUMERIC(14, 2) (Default=0, Nullable=False). Scheme discount deduction.
- tax_amount: NUMERIC(14, 2) (Default=0, Nullable=False). GST line tax.
- line_total: NUMERIC(14, 2) (Nullable=False). Net line total.
- Check Constraint (ck_order_item_amounts_nonnegative): unit_price >= 0 AND discount_amount >= 0 AND tax_amount >= 0 AND line_total >= 0.

**18. Table: order_status_history**

- id: UUID (Primary Key).
- order_id: UUID (Foreign Key referencing orders.id ondelete CASCADE, Indexed).
- previous_status: VARCHAR(30) (Nullable=True).
- new_status: VARCHAR(30) (Nullable=False).
- changed_by: UUID (Foreign Key referencing users.id ondelete RESTRICT, Nullable=True).
- note: TEXT (Nullable=True). Audit comment explaining state change (e.g. "Credit limit cleared by desk").
- created_at: DateTime(timezone=True) (Server Default func.now()).

# DESIGN SYSTEM & VISUAL HIERARCHY

1. The Restrained Minimalist Palette:

- Background: Warm off-white canvas (#FBF9F1, #F3F0E6) rather than harsh sterile white (#FFFFFF). This reduces eye fatigue during extended 45-minute inventory reviews and improves visibility under outdoor sunlight.
- Borders: Warm slate framing (#DCD9D0) creating distinct card boundaries without visual clutter.
- Typography: Deep charcoal (#242724) for primary headings and prices; muted slate (#79565D, #64748B) for secondary pack specifications.
- Status Accents: Emerald green (#2D7A4F) for approved orders and active schemes; warm amber (#D97706) for credit holds and disambiguation cards; crimson (#DC2626) for rejected or out-of-stock items.

2. Touch Ergonomics for Single-Handed Operation:

- Every interactive element (quantity steppers, unit selector pills, camera triggers, and checkout buttons) enforces a minimum height of 48px to 52px.
- This accommodates rapid, one-handed thumb navigation while standing at a physical shop counter with a customer waiting.

3. Fixed Navigation Architecture:

- Both the top header (fixed top-0 left-0 right-0 h-14 z-40) and the sidebar navigation (fixed top-14 left-0 bottom-0 w-60 z-30) remain permanently pinned to the browser viewport.
- The content area uses explicit offset padding (pt-14 md:pl-60). This ensures that when a merchant scrolls through hundreds of SKUs, the search bar, active category tabs, and floating cart badge never scroll out of view.

## KEY SCREENS & DESIGN RATIONALE

### Screen 1: Kirana Product Catalogue (/retailer/catalogue)

- Why it looks this way:
  - High-density SKU cards featuring large, recognizable product packaging imagery alongside bold wholesale pricing.
  - Transparent GST Inclusion: Every product card displays explicit tax indicators (e.g. "₹8.10/pc incl. 18% GST" alongside case prices). In wholesale distribution, unexpected tax additions at the time of delivery trigger refused shipments; showing tax-inclusive figures upfront builds trust.

○ Interactive Unit Toggle: A tactile segmented control allows merchants to switch between "पेटी / Case (96 pcs)" and "पीस / Piece" with instant price recalculation.
○ Clear Out-of-Stock Signaling: SKUs with zero inventory (e.g. Tea Leaf 250g) are visually dimmed with an explicit "Out of Stock" pill, disabling the add-to-cart stepper to prevent post-ordering line cuts.

**Screen 2: Cart with Dynamic Wholesale Scheme Preview (/retailer/cart)**

● Why it looks this way:
○ Clean separation between standard ordered items, automatically earned free goods, and invoice deductions.
○ Zero-Math Free Goods Display: When a retailer orders 10 cases of Glucose Biscuits, the cart automatically injects a distinct green card: "+1 Case Free Goods (Buy 10 Get 1 Applied)" at ₹0.00. Shopkeepers do not have to calculate promotional entitlements manually.
○ Volume Slab Discount Pill: Order lines meeting volume thresholds (e.g. 5+ cases of detergent) highlight the active discount rate ("3% Slab Discount Applied - You saved ₹304.50").
○ Credit Limit Progress Gauge: A real-time balance bar shows current credit exposure, newly added order total, and remaining available credit headroom before checkout is initiated.

**Screen 3: Conversational AI Ordering Assistant (/retailer/assistant)**

● Why it looks this way:
○ Split-view interface: The left panel provides a continuous conversational message stream (supporting both text typing and microphone voice notes); the right panel maintains a live, server-authoritative draft order summary.
○ Interactive Yellow Disambiguation Cards: When the AI encounters ambiguous requests (e.g. "bhaiya 2 chhota biscuit"), it renders a prominent amber card containing:
1. The verbatim phrase spoken by the shopkeeper.
2. The calculated match confidence percentage (e.g. 74%).
3. Two alternative product buttons (Glucose 75g vs Marie 120g).
4. An explicit unit question: "Did you mean 2 Peti (Cases) or 2 Packets?".
○ Human-in-the-Loop Confirmation: The interface makes it visually obvious that the AI only creates draft items. An order is submitted to the warehouse only when the shopkeeper taps "Confirm Wholesale Order".

**Screen 4: Order Desk Triage & Operations Console (/order-desk)**

● Why it looks this way:
○ Tabular triage queue displaying aggregated status counters across top KPI cards (All, Submitted, On Hold, Approved, Dispatched).
○ Prominent Credit Hold Flagging: Orders placed on hold are highlighted with amber warning badges and descriptive reason tags (e.g. "Credit Limit Exceeded - Exposure: ₹64,250 / Limit: ₹50,000").

○ Rapid Batch Actions: Operators can filter by beat territory or salesman, inspect line items in a side drawer without losing table context, and trigger one-click status transitions.

**Screen 5: Consolidated Van Loading Sheet (/order-desk/van-loading)**

- Why it looks this way:
  - ○ Grouped picking list formatted specifically for warehouse floor workers and delivery truck loaders.
  - ○ Aggregates hundreds of individual retail orders into consolidated brand and SKU totals (e.g. "Brand: Parle -> Glucose Biscuit 75g: 34 Cases Total across 18 Orders").
  - ○ High-contrast, printable typography ready for physical clipboards on the warehouse loading dock.

**Screen 6: Field Salesman Beat Portal (/salesman)**

- Why it looks this way:
  - ○ Mobile-first, card-based layout organized by geographic market beat (Wright Town, Napier Town, Sadar Bazaar).
  - ○ Store Visit Context: Each kirana card displays the owner name, physical address, current outstanding credit balance, and days since last order.
  - ○ Offline-Ready Indicator: A persistent status chip indicates whether the salesman is online or operating in offline-cached queue mode.

## WHAT WAS CUT, AND WHAT WE WOULD DO NEXT

### What Was Cut (Intentional Scope Boundaries & Trade-offs)

To deliver a reliable, deeply functional core product within the 7-day solo development timeframe, the following features were deliberately excluded based on engineering trade-offs:

**1. Delivery Vehicle GPS Fleet Tracking:**

- Why it was cut: Real-time driver GPS tracking requires mobile driver hardware, continuous cellular background location pings, and map rendering engines. In regional FMCG distribution, delivery routes are fixed daily beats where trucks follow predetermined morning routes.
- What was built instead: Consolidated Van Loading Sheets (/order-desk/van-loading) that solve the immediate commercial bottleneck—getting trucks packed and dispatched accurately before 8:00 AM.

**2. In-App Consumer Digital Payments (UPI / Cards):**

- Why it was cut: B2B wholesale transactions in Jabalpur average ₹5,000 to ₹25,000 per invoice. Retailers do not pay via credit cards or consumer UPI apps at checkout; settlements occur via rolling 14-day credit ledgers settled through weekly cash collections, account transfers, or post-dated cheques.
- What was built instead: A double-entry credit ledger system (credit_accounts and credit_transactions) with automated credit limit enforcement and hold queues.

**3. Direct Statutory GST Portal E-Invoicing:**

- Why it was cut: Regional FMCG distributors already maintain legal statutory compliance and e-invoicing accounts inside desktop Tally ERP. Rebuilding an e-way bill generation pipeline in a custom app would duplicate statutory logic and introduce legal tax filing liability.
- What was built instead: Compliant Tally ERP9 / Tally Prime XML export batches that allow back-office operators to ingest approved orders into Tally with a single click.

**4. Binary Streaming WebSockets for Audio:**

- Why it was cut: Maintaining long-lived bidirectional WebSocket binary audio streaming over low-tier Indian 3G networks suffers from frequent socket disconnects and high battery drain on budget Android devices.
- What was built instead: Browser-side Web Audio API bandpass pre-filtering combined with resilient RESTful NLU payloads and Google Gemini Vision slip OCR, backed by deterministic offline regex fallbacks.

**What We Would Do Next**

Orderly provides a foundation for digitizing distributor-retailer operations through centralized order management, automated order processing, inventory visibility, pricing, and AI-assisted order capture. The system can be further enhanced with the following features to improve automation, accessibility, scalability, and operational efficiency.

## 1 WhatsApp Business Integration

The current order-capture workflow can be extended through the WhatsApp Business Platform (Meta Cloud API). Retailers could place orders directly by sending text messages, product-list images, or voice notes through WhatsApp.

The system could process incoming messages using AI, identify products and quantities, calculate applicable schemes, and generate a draft order. Retailers would receive an order summary for confirmation before submission.

## 2 Advanced Offline Support for Salesmen

The salesman portal can be enhanced to support reliable order placement in areas with limited or unstable internet connectivity.

- Store essential product, retailer, and pricing information locally on the device.
- Allow salesmen to create and save orders while offline.
- Synchronize pending orders automatically when connectivity is restored.
- Detect data conflicts and provide appropriate resolution options.
- Display synchronization status to prevent duplicate or incomplete submissions.

This would make the application more dependable for salesmen operating in areas with poor network coverage.

## 3 Native Android Application

Orderly can be extended into a dedicated Android application to provide a more convenient mobile experience for retailers and salesmen.

The application could support device-level features such as push notifications, improved camera access for product-list scanning, local data storage, and background synchronization. This would improve accessibility for users who primarily operate through smartphones.

## 4 Predictive Inventory and Replenishment

Historical order data can be used to develop predictive analytics capabilities that help distributors and retailers make better inventory decisions.

Potential enhancements include:

- Forecasting product demand based on historical ordering patterns.
- Identifying frequently ordered and slow-moving products.
- Predicting approximate stock depletion dates.
- Generating low-stock alerts and suggested reorder quantities.
- Identifying seasonal changes in product demand.

These capabilities could help reduce stock shortages and improve inventory planning.

## 5 Two-Way Tally ERP Integration

The existing Tally export functionality can be expanded into a two-way integration with Tally ERP.

Future integration could support:

- Automatic transfer of approved orders into Tally.
- Synchronization of invoices and payment records.
- Updating inventory information between both systems.
- Retrieving relevant accounting and transaction details.
- Maintaining synchronization logs and handling failed transactions.

This would reduce manual data entry and improve consistency between order management and accounting operations.

## 6 Advanced Business Analytics and Reporting

The distributor dashboard can be enhanced with advanced reporting and data visualization to provide deeper insights into business performance.

Possible additions include:

- Retailer-wise and product-wise sales analysis.
- Monthly and seasonal revenue trends.
- Scheme performance and discount analysis.
- Outstanding credit and payment trend reports.
- Salesman performance and territory-wise comparisons.
- Identification of inactive retailers and declining order frequency.

These insights could support more informed business decisions and improve distributor operations.

## 7 Multi-Warehouse Inventory Management

The inventory module can be expanded to support complete multi-warehouse operations.

Distributors could manage stock across multiple warehouses, monitor warehouse-specific availability, and allocate orders based on stock levels and delivery locations. Automated stock transfers and warehouse-level reporting could further improve inventory management.

# TECHNOLOGY STACK

The technology stack was selected to satisfy the operational realities of regional FMCG distribution: high I/O throughput, strict transactional consistency, low-latency mobile rendering on 3G connections, and zero-downtime AI fallbacks.

**TECHNOLOGY MATRIX AND SELECTION JUSTIFICATION**

| **Architectural Layer** | **Technology & Version** | **Purpose in Orderly** | **Justification & Verification Status** |
|---|---|---|---|
| Frontend Framework | Next.js 16.4.4 (App Router, Turbopack) | Core web presentation layer | Verified: Server-rendered static routing minimizes client JavaScript payload, achieving under 3 seconds first-load latency on low-end Android mobile devices over 3G. |
| Frontend Runtime | React 19.3.0 | Reactive UI component rendering | Verified: Provides fine-grained concurrent rendering and optimized hook lifecycles across dynamic order forms, cart recalculations, and conversational chat streams. |
| Styling & Design System | Tailwind CSS v4 & Base UI (@base-ui/react) | Design system and touch layout | Verified: Utility-first styling with zero runtime CSS-in-JS overhead; customized for 48px+ touch targets and high-contrast retail counter environments. |
| Iconography | Lucide React 1.67.0 | UI visual indicators | Verified: Lightweight tree-shakeable SVG icons for status badges, navigation, and stock indicators. |
| Programming Languages | TypeScript 5 (Frontend) & Python 3.11/3.13 (Backend) | Full-stack type safety | Verified: TypeScript guarantees end-to-end interface integrity on the web; Python provides native interoperability with modern AI APIs and data manipulation libraries. |

| | | | |
|---|---|---|---|
| Backend Framework | FastAPI 0.115+ | Asynchronous REST API Gateway | Verified: Native async ASGI architecture handles concurrent order submissions; automated OpenAPI schema generation; built-in dependency injection for RBAC. |
| Database ORM | SQLAlchemy 2.0.38 | Object-Relational Mapping | Verified: Modern Mapped typing, transactional session management, explicit foreign key cascades, and row-level locking (with_for_update) for credit checks and inventory. |
| Database Migrations | Alembic 1.15.1 | Schema version control | Verified: Declarative migration scripts ensure repeatable database evolutions across local PostgreSQL and cloud Neon environments. |
| Primary Database | PostgreSQL 16 (Neon Serverless) | Relational data persistence | Verified: Strict ACID transaction guarantees, multi-table foreign key constraints, numeric precision storage, and pooled SSL connections. |
| Authentication Service | Firebase Authentication & Admin SDK | Identity verification | Verified: Client-side token generation mapped on the backend via Firebase Admin SDK; Firebase UIDs bound to PostgreSQL user records. |
| AI NLU Engine | Groq API (qwen/qwen3.8-27b / Llama 3.3) | Conversational entity extraction | Verified: Sub-450ms inference via Groq LPUs; structured JSON function-calling schema extracting products, quantities, and Hindi units. |
| AI Vision OCR Engine | Google Gemini 2.5 Flash | Handwritten parchi transcription | Verified: Multimodal image recognition parsing mixed Hindi/English handwritten order chits, with API key rotation fallback across GEMINI_API_KEY and GEMINI_API_KEY1. |
| Media Cloud Storage | Cloudinary | Product and slip image storage | Verified: Offloads binary image hosting from database; generates secure CDN-optimized asset URLs. |

| | | | |
|---|---|---|---|
| Testing Framework | Pytest 9.2.0 & pytest-asyncio | Automated testing suite | Verified: 88 automated tests verifying NLU matchers, scheme rules, price fallbacks, credit limits, and full end-to-end flows. |
| Backend Deployment | Render Web Service (Python 3.11) | Cloud container hosting | Verified: Live deployment running at https://orderly-w9x4.onrender.com with automated continuous deployment from GitHub main branch. |
| Frontend Deployment | Vercel Edge Global Network | Edge CDN hosting | Verified: Automatic CI/CD build from GitHub main directory, global edge caching, and zero-configuration Next.js SSR support. |